

Database Design

DB Overview

Tables:

“Customer” - Create a table named "Customer" to store customer information. The table has four columns: cust_id (primary key, positive integer), full_name (non-null text), phone (non-null text), and address (non-null text with a minimum length of 3 characters). The function sets up the structure for storing essential customer data with basic data integrity checks.

“Orders” - Create a table named "Order" to store order information. The table has two columns: order_id (primary key, positive integer) and date (non-null timestamp without time zone).

“Dish” - Create a table named "Dish" to store dish information. The table has four columns: dish_id (primary key, positive integer), name (non-null text with minimum trimmed length of 3), price (non-null positive decimal), and is_active (non-null boolean).

“Customer Orders” - The Customer_Orders table links customers to their orders, creating a one-to-many relationship where a customer can have multiple orders. This table uses foreign keys to maintain referential integrity, with cascading deletes to ensure that orders are removed if the associated customer is deleted.

“Dishes in Order” - The Dishes_in_Order table captures the relationship between orders and dishes, detailing the amount and price of each dish within an order. This setup allows for comprehensive tracking of order contents and pricing. The primary key constraint ensures that each order-dish pair is unique, while checks on order_id, dish_id, and amount enforce data integrity.

“Likes” - The Likes table records which dishes are liked by customers, supporting a many-to-many relationship between customers and dishes. The primary key on (cust_id, dish_id) ensures that each customer can only like a dish once, and foreign keys maintain data consistency with the Customer and Dish tables.

Views:

“OrderTotalPrice” - Create a view named "OrderTotalPrice" that calculates the total price of each order. It joins the "Order" and "Dishes_in_Order" tables, summing the product of price and amount for each order, with a default of 0 for orders without dishes. This view simplifies the process of retrieving order totals for reporting and analysis.

“MostLikedDishes” - The view provides a summary of the number of likes each dish has received. It aggregates data from the "Likes" table, which records the dishes that customers have liked. Each row in this view corresponds to a unique dish, identified by dish_id, along with a count of how many times that dish has been liked. This view is useful for identifying the most popular dishes based on customer preferences.

"MostPurchasedDish" - The view calculates the total number of times each dish has been purchased across all orders. It pulls data from the "Dishes_in_Order" table, which records the quantity of each dish ordered. The view aggregates these amounts to provide a total_sum for each dish, representing the total purchases. Each row corresponds to a unique dish identified by dish_id. This view helps in identifying the dishes with the highest sales volume.

Functions:

ReturnValue add_customer(customer: Customer)

Add a customer to the "Customer" table.

Customer get_customer(customer_id: int)

Get a customer from the "Customer" table.

ReturnValue delete_customer(customer_id: int)

Delete a customer from the "Customer" table.

ReturnValue add_order(order: Order)

Add an order to the "Order" table.

Order get_order(order_id: int)

Get an order from the "Order" table.

ReturnValue delete_order(order_id: int)

Delete an order from the "Order" table.

ReturnValue add_dish(dish: Dish)

Add a dish to the "Dish" table.

Dish get_dish(dish_id: int)

Get a dish from the "Dish" table.

ReturnValue update_dish_price(dish_id: int, price: float)

Update the price of the dish in "Dish" table with the given dish_id.

ReturnValue update_dish_active_status(dish_id: int, is_active: bool)

Update the active status of the dish in "Dish" table with the given dish_id.

ReturnValue customer_placed_order(customer_id: int, order_id: int)

Customer specified by customer_id has placed order with the given order_id. Inserts the customer_id and order_id to the "Customer_Orders" table.

Customer get_customer_that_placed_order(order_id: int)

Get the customer who placed the order with the given order_id from the JOIN of "Customer_Orders" table and "Customer" table.

ReturnValue order_contains_dish (order_id: int , dish_id: int, amount: int)

Add the dish specified by dish_id to the order specified by order_id, along with the amount ordered and the dish's price at the time of ordering. Inserts the parameters and price of the dish into "Dishes_in_Order" table, ensuring that the dish is active by using subqueries to check the dish's existence and retrieve its price from the "Dish" table.

ReturnValue order_does_not_contain_dish (order_id: int , dish_id: int)

Remove the dish with the given dish_id from the order with the given order_id. If the parameters are valid, it deletes the corresponding record (dish_id and order_id) from the "Dishes_in_Order" table.

List[OrderDish] get_all_order_items(order_id: int)

Get a list of all the dishes that the order with the given order_id contains.

Selects the dish_id, amount, and price of each dish in the specified order from the "Dishes_in_Order" table, joining it with the "Dish" table for additional details. The results are ordered by dish_id in ascending order.

ReturnValue customer_likes_dish(cust_id: int, dish_id: int)

Customer with the given cust_id likes the dish with the given dish_id. Inserts the parameters into "Likes" table.

ReturnValue customer_dislike_dish(cust_id: int, dish_id: int)

Customer with the given cust_id dislikes the dish with the given dish_id and does not like it anymore. If the parameters are valid, it removes the corresponding record (cust_id and dish_id) from the "Likes" table.

List[Dish] get_all_customer_likes(cust_id: int)

Get a list of all the dishes that customer likes. get the dishes liked by the customer with the given cust_id from the JOIN of "Likes" table and "Dish" table, ordered by dish_id in ascending order.

float get_order_total_price(order_id: int)

Get the total price of the order specified by order_id.

Selects the total price from the "OrderTotalPrice" view for the given order_id. If the query returns no results, the function sets the total price to 0.0.

float get_max_amount_of_money_cust_spent(cust_id: int)

Get the maximum amount of money the customer with the given cust_id has ever spent in

one order. get the highest total price of orders placed by the customer with the given cust_id from the JOIN of "OrderTotalPrice" view and "Customer_Orders" table. If the query returns no results, the function sets the max to 0.0.

Order get_most_expensive_anonymous_order()

Get the order with the maximum total price among all the orders that are not related to any customer. Calculates the total price of each anonymous order by summing the product of the price and amount of each dish included in the order. The query joins the "Order" table with the "Customer_Orders" and "Dishes_in_Order" tables to filter out non-anonymous orders and aggregate the total price. The results are ordered by total price in descending order and by order_id in ascending order, selecting the top result.

bool is_most_liked_dish_equal_to_most_purchased()

Returns True if the most liked dish is also the most purchased, else returns false. Check if the most liked dish is also the most purchased dish by comparing the dish_id from the "MostLikedDishes" view (taking the first row after ordering by dish_id and number of likes) with the dish_id from the "MostPurchasedDish" view (taking the first row after ordering by number of purchased and dish_id).

List[int] get_customers_ordered_top_5_dishes ()

Get all customers who have ordered all the dishes in the 5 most liked dishes.

Identifies the top 5 dishes based on their presence in the "MostLikedDishes" view and if there are no 5 dishes, complete from "Dish" table. Then checks the join between "Customer_Orders" and "Dishes_in_Order" tables to find customers who have ordered these dishes. The query ensures that only customers who have ordered all 5 of these dishes are included.

List[int] get_non_worth_price_increase()

Get all active dishes ids where the current price results in a lower average profit per order compared to a previous (lower) price. Calculates the current profit of each active dish by averaging the amount of dishes sold multiplied by their current price. It then calculates the maximum previous profit for each dish by averaging the amount sold at different previous prices. The function compares the current profit with the maximum previous profit and selects the dishes where the current profit is lower.

List[Tuple[int, float]] get_total_profit_per_month(year: int)

Calculate the total profit per month for a specific year across all orders. Get a list of tuples containing the month number and total profit for each month in the given year. Calculates the profit for each month by summing the product of amount and price for all dishes in orders. It uses a LEFT JOIN between the "Order" and "Dishes_in_Order" tables to include months with no orders (assigning 0 profit). The function extracts the month from the order date, groups results by month, and orders them from latest to earliest month of the year.

List[int] get_potential_dish_recommendations(cust_id: int)

Get all dish IDs that might be of interest to a given customer based on the dishes liked by similar customers. Get a list of dish IDs recommended for the customer with the given cust_id. Identifies similar customers who like at least 3 of the same dishes as the given customer. Then, it selects dishes liked by these similar customers but not yet liked by the given customer. The function uses JOINS between "Dish", "Likes", and "Customer" tables to gather this information. It excludes dishes already liked by the given customer and orders the results by dish_id in ascending order.

void createTables()

Creates the tables and views as described above in the implementation section.

void clearTables()

Clears the tables and views.

void dropTables()

Drops the tables and views from the DB.